



CEBU INSTITUTE OF TECHNOLOGY
U N I V E R S I T Y

IT342-Section SYSTEMS INTEGRATION AND ARCHITECTURE 1

FUNCTIONAL REQUIREMENTS SPECIFICATION (FRS)

Project Title: **Mini App – User Registration & Authentication**

Prepared By: **Jascha B. Base**

Date of Submission: **02/03/2026**

Version: **0.0.1**

Table of Contents

- 1. Introduction.....3
 - 1.1. Purpose..... 3
 - 1.2. Scope..... 3
 - 1.3. Definitions, Acronyms, and Abbreviations..... 3
- 2. Overall Description.....3
 - 2.1. System Perspective..... 3
 - 2.2. User Classes and Characteristics.....3
 - 2.3. Operating Environment..... 3
 - 2.4. Assumptions and Dependencies..... 3
- 3. System Features and Functional Requirements.....3
 - 3.1. Feature 1:.....3
 - 3.2. Feature 2:.....3
- 4. Non-Functional Requirements..... 3
- 5. System Models (Diagrams)..... 4
 - 5.1. ERD..... 4
 - 5.2. Use Case Diagram..... 4
 - 5.3. Activity Diagram.....4
 - 5.4. Class Diagram.....4
 - 5.5. Sequence Diagram.....4
- 6. Appendices.....4

1. Introduction

1.1. Purpose

The purpose of this document is to provide a detailed overview of the User Authentication and Profile Management System. It serves as a blueprint for me and the stakeholders to understand the functional and non-functional requirements of the system, ensuring secure user access and data persistence.

1.2. Scope

The system will provide a secure gateway for users to access protected application features. It covers **User Registration, Secure Login, Profile Dashboard** viewing, and **Session Termination** (Logout).

1.3. Definitions, Acronyms, and Abbreviations

API (Application Programming Interface): The set of rules allowing the React frontend to talk to the Spring Boot backend.

2. Overall Description

2.1. System Perspective

It follows a client-server architecture where the React UI handles user interaction and the Spring Boot API manages business logic and database communication.

2.2. User Classes and Characteristics

Guest User: Unauthenticated individuals who can only access the landing, registration, and login pages.

Authenticated User: Logged-in individuals with a valid token who can access the dashboard and their personal profile data.

2.3. Operating Environment

Frontend: React.js (Node.js environment), running on modern web browsers

Backend: Spring Boot (Java JDK 17+), running on an application server.

Database: Relational Database (PostgreSQL or MySQL).

Tools: VS Code/IntelliJ IDEA, Postman for API testing.

2.4. Assumptions and Dependencies

Assumption: Users have a stable internet connection to communicate with the API.

Dependency: The system depends on the availability of the Database server for user credential verification.

3. System Features and Functional Requirements

Describe each major feature of the system and its functional requirements.

3.1. Feature 1:

Feature 1: User Identity Management (Registration & Login)

Description: Provides the entry point for users to establish and verify their identity within the system.

Functional Requirements:

Account Creation: The system shall allow guests to register by providing a unique username, email, and password.

Credential Hashing: The system must hash passwords before database storage to ensure security.

Identity Verification: The system shall grant access only if the provided credentials match the stored record.

3.2. Feature 2:

Feature 2: Access Control & Profile Management

Description: Manages the user's session and protects specific pages from unauthorized access.

Functional Requirements:

Session Authorization: The system shall issue a secure token (JWT) upon login to authorize access to the user's dashboard.

Page Protection: The system shall prevent unauthenticated users from accessing protected profile pages.

Secure Logout: The system shall clear the authentication token upon a logout request.

4. Non-Functional Requirements

Attribute	Requirement
Security	Protects against unauthorized access by using industry-standard hashing and token-based authentication.
Performance	Authentication and profile loading should occur within 500ms to ensure a responsive user experience.

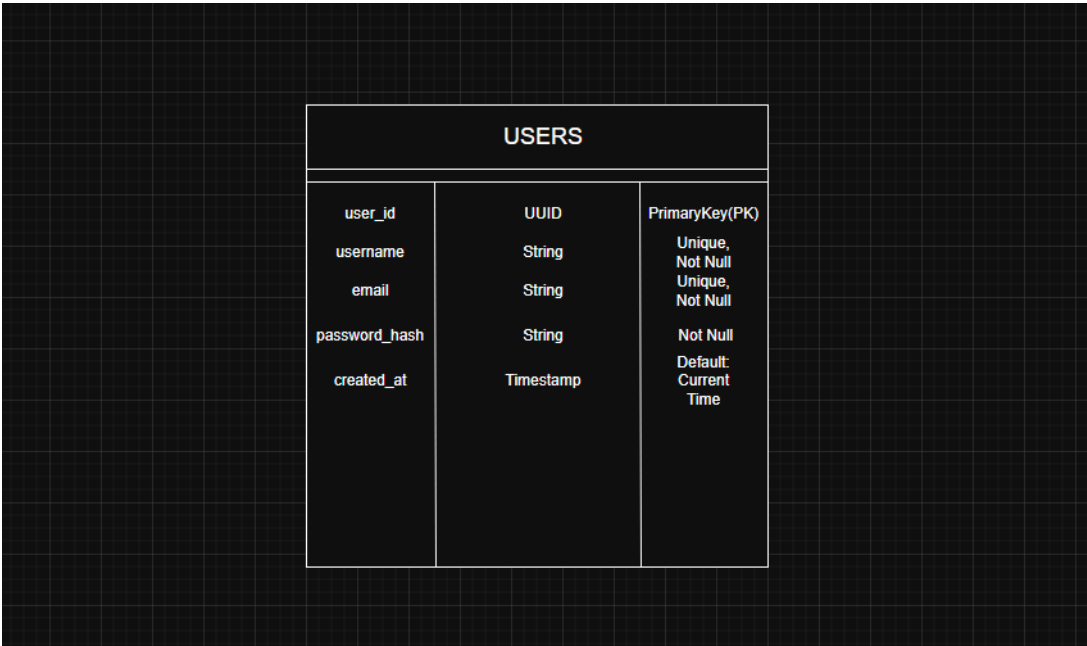
Usability	The interface should utilize a clean layout with consistent typography like Montserrat or Playfair Display
Reliability	The database and API must maintain high availability to prevent users from being locked out.

5. System Models (Diagrams)

Insert the necessary diagrams for the system:

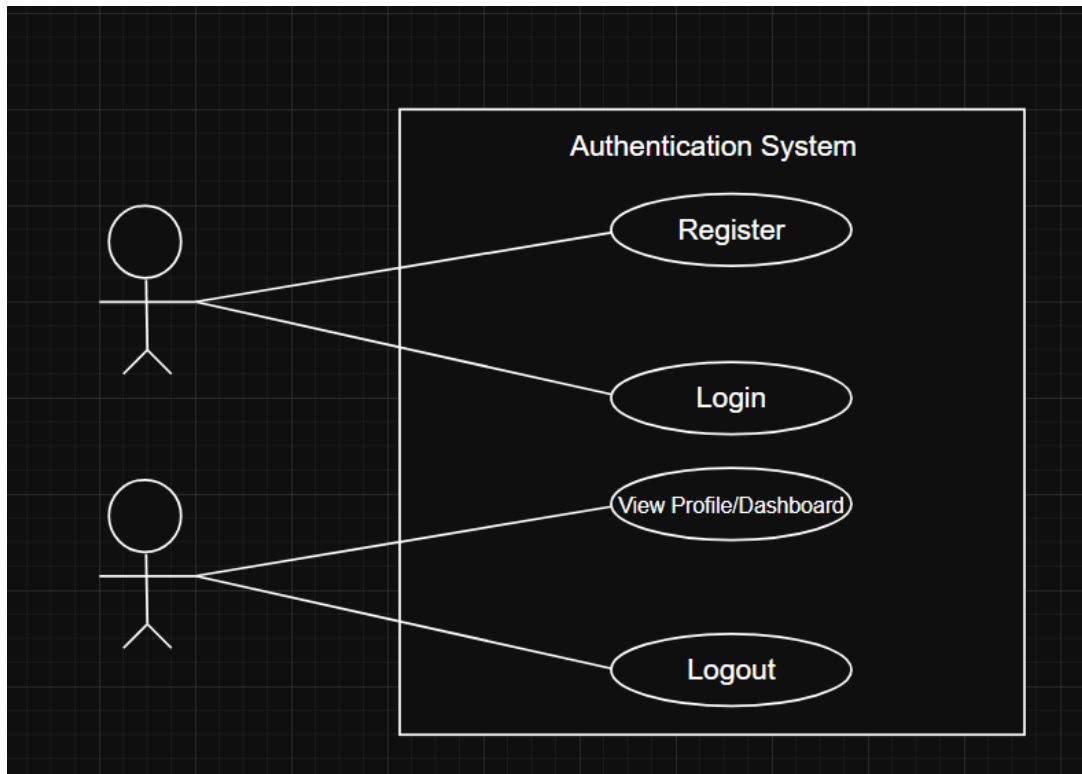
5.1. ERD

ERD



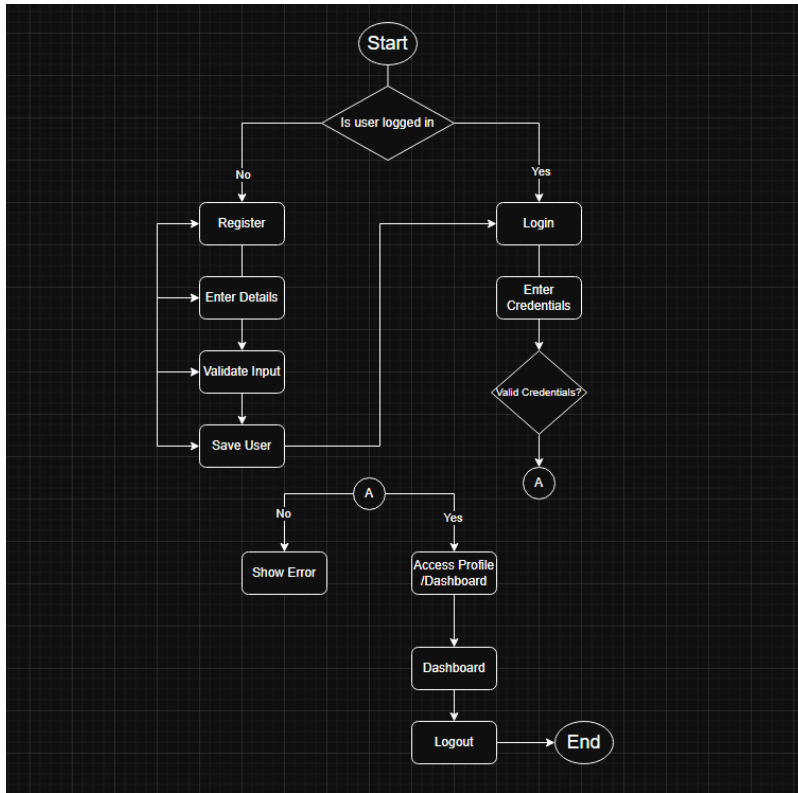
5.2. Use Case Diagram

USER CASE DIAGRAM



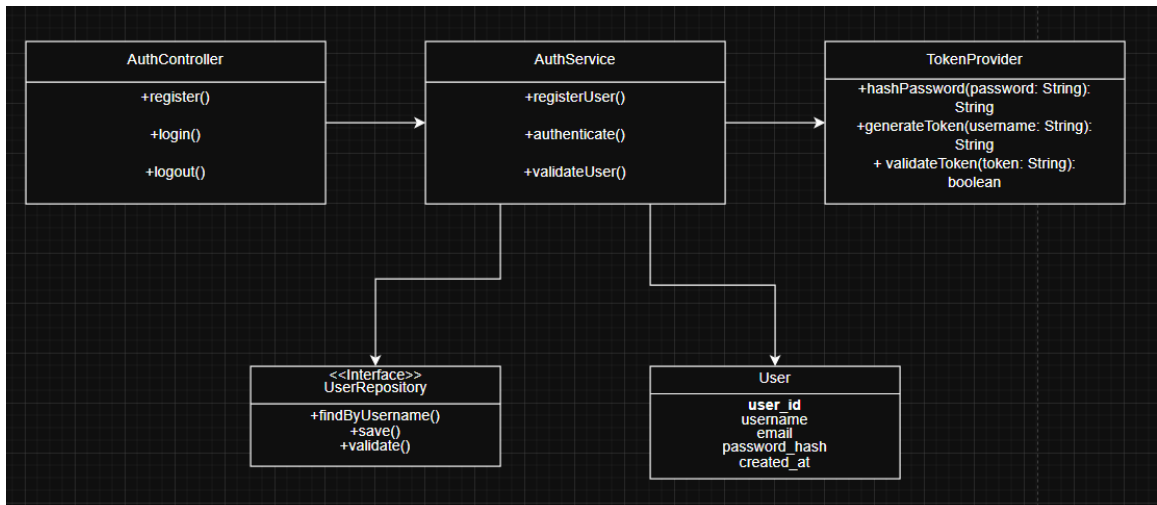
5.3. Activity Diagram

ACTIVITY DIAGRAM



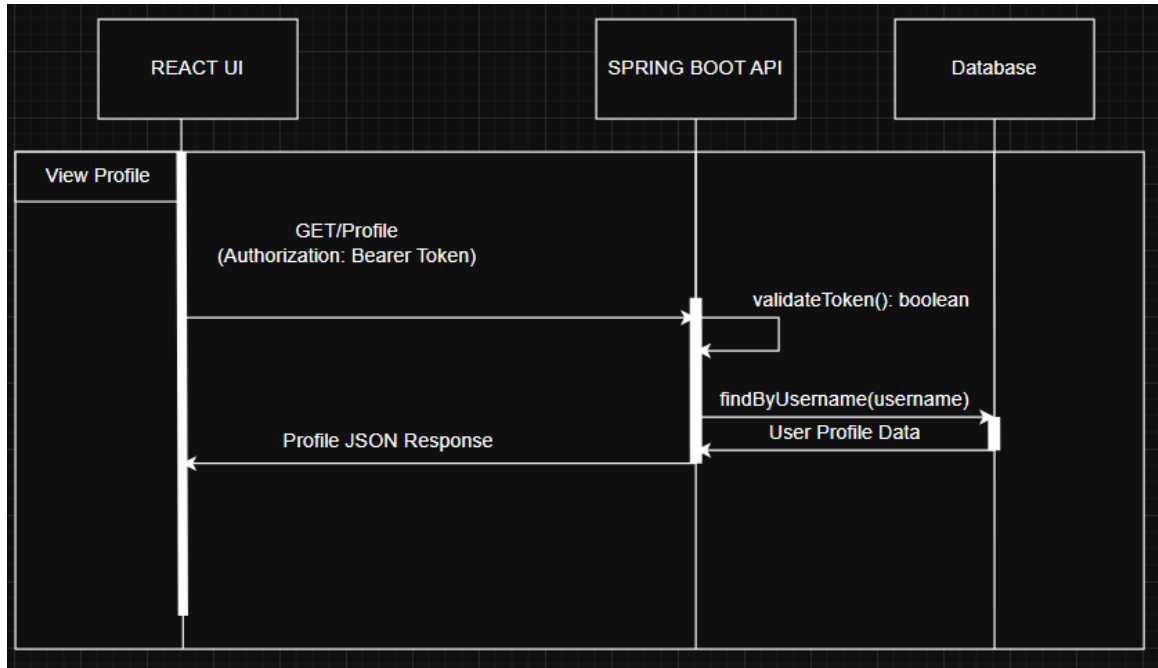
5.4. Class Diagram

CLASS DIAGRAM



5.5. Sequence Diagram

SEQUENCE DIAGRAM



6. Appendices

Include any additional information, references, or support materials.